



ACADEMIC CITY UNIVERSITY

Faculty of Computational Sciences and Informatics

END OF MODULE EXAMINATION PROJECT

CS3231 – Computer Graphics and Multimedia Systems

Mini 3D Game Environment

(Option A: Interactive 3D Scene Simulator)

Team Members

Name

Albert Afful
Yoofi Egyin Brew-Hammond
Jeremy Bradley Aryee Arthur
Amazing Naa Dedei Quaye
Edinam Kumordjie
Benjamin Acheampong
Stanley Agambire
Bayaka Melvin Soale Braimah
Samuel Nii Ayitey Aryee
Erica Dzifa Agbana

Course: CS3231 – Computer Graphics and Multimedia Systems
Lecturer: Fredrick Boafo
Submission Date: 29th April 2026
Maximum Marks: 100

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Technology Stack	3
2	System Architecture	4
2.1	Game Mode Setup	4
2.2	Level and World Setup	5
2.3	Blueprint Class Hierarchy	5
3	3D Modelling and Character Setup	7
3.1	Character Mesh	7
3.2	Weapon Socket and Attachment	7
3.3	AR4 Weapon Component	8
4	Animation System	10
4.1	Mixamo Animation Sourcing	10
4.2	Animation Retargeting	10
4.3	Animation Blueprint (ABP_Survival)	11
4.3.1	Animation Variables	12
4.4	Aim Animation Sequence	12
4.5	Running Animation with MetaSound Notifiers	13
5	Blueprint Scripting and Interaction	15
5.1	Aim-Down-Sights (ADS) Mechanic	15
5.2	Crouch Mechanic	16
5.3	Enemy AI Blueprint	16
6	Colour Models, Rendering and Visual Effects	18
6.1	Particle System – Muzzle Flash	18
6.1.1	Emitter Configuration	19
6.2	Level Environment and Lighting	19
7	User Interface Design	20
7.1	Player Health Bar (WB_HB)	20
7.2	Enemy Health Bar (WB_Enemy_HB)	20
7.3	Crosshair Widget (WB_Crosshair)	22

8 Multimedia Integration	23
8.1 Audio System – MetaSound	23
8.1.1 Audio Architecture	24
8.1.2 Compression and Format Justification.....	24
8.2 Video and Animation Integration.....	24
9 Viewing, Clipping and Camera Control.....	25
9.1 Third-Person Camera Setup.....	25
9.2 Window-to-Viewport Mapping	25
9.3 Scene Clipping.....	25
10 2D and 3D Transformations	26
10.1 Skeletal Mesh Transformations.....	26
10.2 Camera Arm Transformation.....	26
10.3 Projection	27
11 Graphics Algorithms	28
11.1 Line Drawing – Line Trace Algorithm	28
11.2 Circle Drawing – Particle Emitters.....	28
11.3 Bézier Curves – Animation Curves	29
12 Team Collaboration	30
12.1 Task Distribution	30
12.2 Academic Integrity	30
13 Conclusion	31
A Screenshot Index	32

Chapter 1

Introduction

This technical report documents the design and implementation of a **Mini 3D Game Environment** built using Unreal Engine 5 as part of the CS3231 Computer Graphics and Multimedia Systems module at Academic City University. The project was developed by a team of ten students over a period of four to five weeks and fulfils the requirements of **Option A: Interactive 3D Scene Simulator**.

1.1 Project Overview

The project is a third-person survival shooter game environment that demonstrates key concepts from the CS3231 course outline, including 3D scene representation, real-time rendering, skeletal animation, particle effects, audio integration, and interactive Blueprint scripting within Unreal Engine 5.

The game features:

- A fully playable third-person character with a military-style character mesh
- Weapon attachment via skeletal socket system
- Aim-down-sights (ADS) camera mechanics
- Crouch and movement animations driven by an Animation Blueprint
- Enemy AI with health tracking and damage logic
- Particle-based muzzle flash effects
- MetaSound-driven audio (footsteps, gunshots)
- On-screen HUD elements: health bar, crosshair, enemy health bar
- A stylised Egypt-themed level environment

1.2 Technology Stack

Technology	Purpose
Unreal Engine 5	3D game environment and rendering
Blueprint Visual Scripting	Game logic and event-driven interaction
Mixamo	Character animations and rigging
MetaSound	Real-time audio graph system
Cascade Particle System	Visual effects (muzzle flash)
UMG (Unreal Motion Graphics)	HUD / UI widget design
Stylized Egypt Asset Pack	Level environment

Chapter 2

System Architecture

2.1 Game Mode Setup

The game uses a custom GM_ThirdPersonShooter Game Mode Blueprint derived from Unreal Engine's base GameModeBase class. The Game Mode defines the core classes used throughout the session.

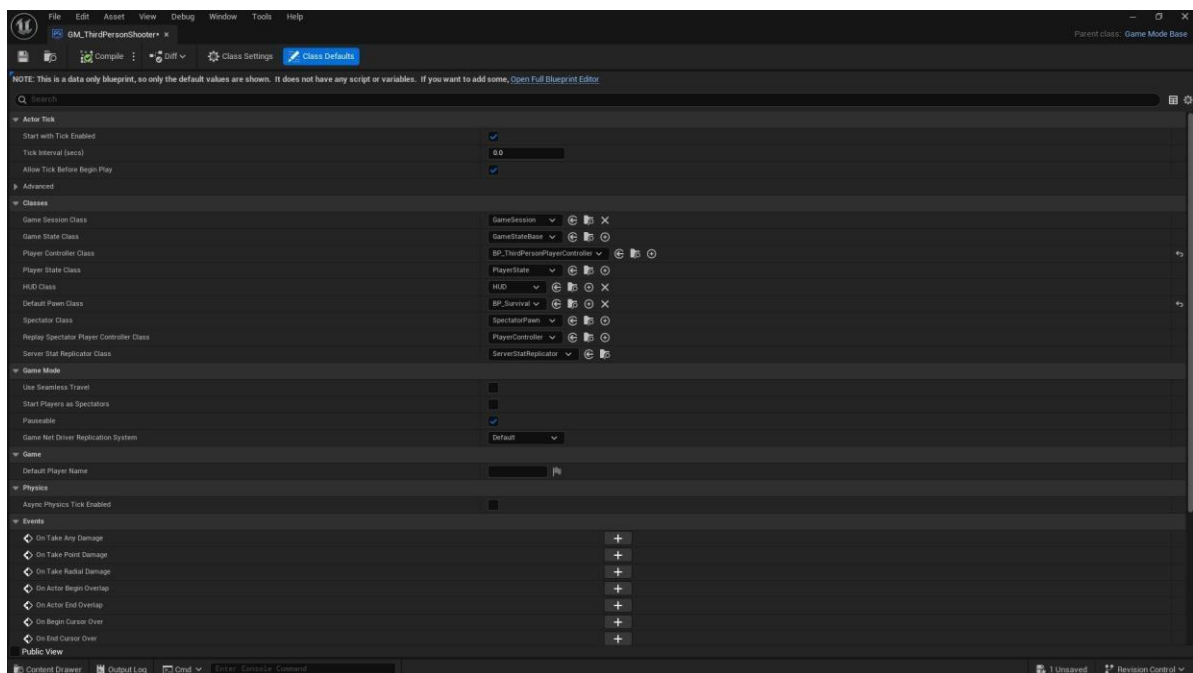


Figure 2.1: GM_ThirdPersonShooter Game Mode Blueprint – Class Defaults panel showing Player Controller (BP_ThirdPersonPlayerController), Default Pawn (BP_Survival), HUD, Game State, and Player State assignments.

Key class assignments visible in Figure 2.1:

- **Player Controller Class:** BP_ThirdPersonPlayerController

- **Default Pawn Class:** BP_Survival (the player character)
- **HUD Class:** HUD
- **Game State Class:** GameStateBase
- **Player State Class:** PlayerState

2.2 Level and World Setup

The primary gameplay level (LvL_ThirdPerson) uses Unreal Engine 5's world partition system. The Game Mode Override in the World Settings is set to GM_ThirdPersonS, which loads all gameplay classes.

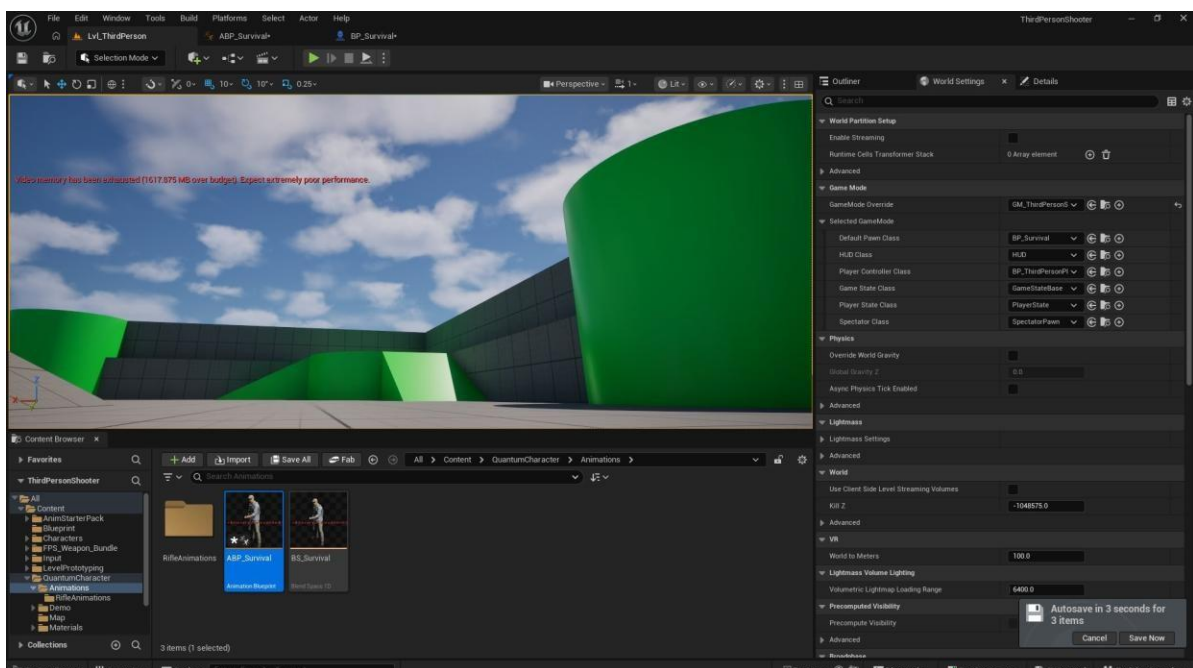


Figure 2.2: Level editor viewport showing the custom 3D environment geometry (green curved architecture). The Content Browser shows the ABP_Survival Animation Blueprint and BS_Survival Blend Space assets. World Settings panel confirms the Game Mode and physics configuration.

2.3 Blueprint Class Hierarchy

The project's Blueprint hierarchy is organised as follows:

Blueprint	Role
BP_Survival	Player character pawn
BP_Survival_Enemy	AI enemy character
ABP_Survival	Player animation blueprint
ABP_Survival_Enemy1	Enemy animation blueprint
GM_ThirdPersonShooter	Game mode
BP_ThirdPersonPlayerController	Player controller
BP_ThirdPersonGameMode	Base game mode

Chapter 3

3D Modelling and Character Setup

3.1 Character Mesh

The player character uses a high-detail military-style skeletal mesh (SKM_QuantumCharacter) featuring 102,882 triangles and 103,477 vertices. The mesh includes UV channels for texture mapping and is rendered at 30 FPS in the preview.

3.2 Weapon Socket and Attachment

A weapon socket named Weapon was created on the hand_r (right hand) bone of the character skeleton. This allows the AR4 rifle to be parented to the hand bone, moving with the character's hand during animations.

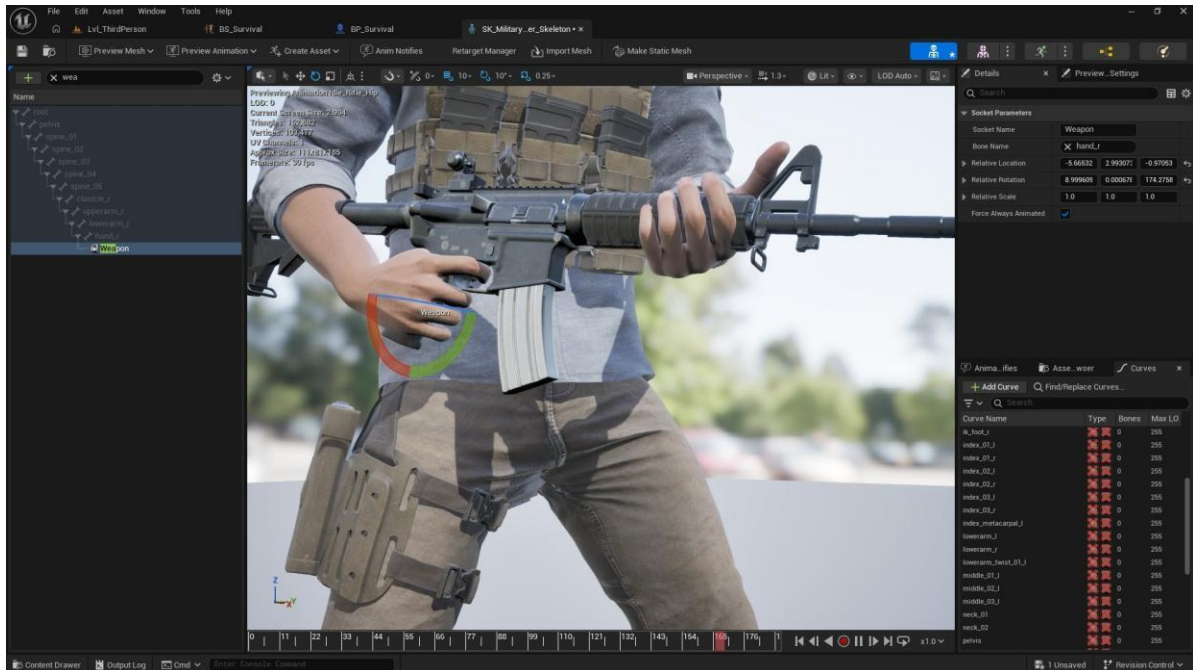


Figure 3.1: Skeleton editor showing the Weapon socket parented to hand_r bone. Socket parameters: Relative Location (-5.66, 2.99, -0.97), Relative Rotation (8.99, 0.0, 174.27). The animation preview shows the Idle_Rifle_Hip animation playing at 30 FPS.

3.3 AR4 Weapon Component

The AR4_Weapon is implemented as a SkeletalMeshComponent child of the BP_Survival character Blueprint. It is attached to the Weapon socket so it follows hand movement.

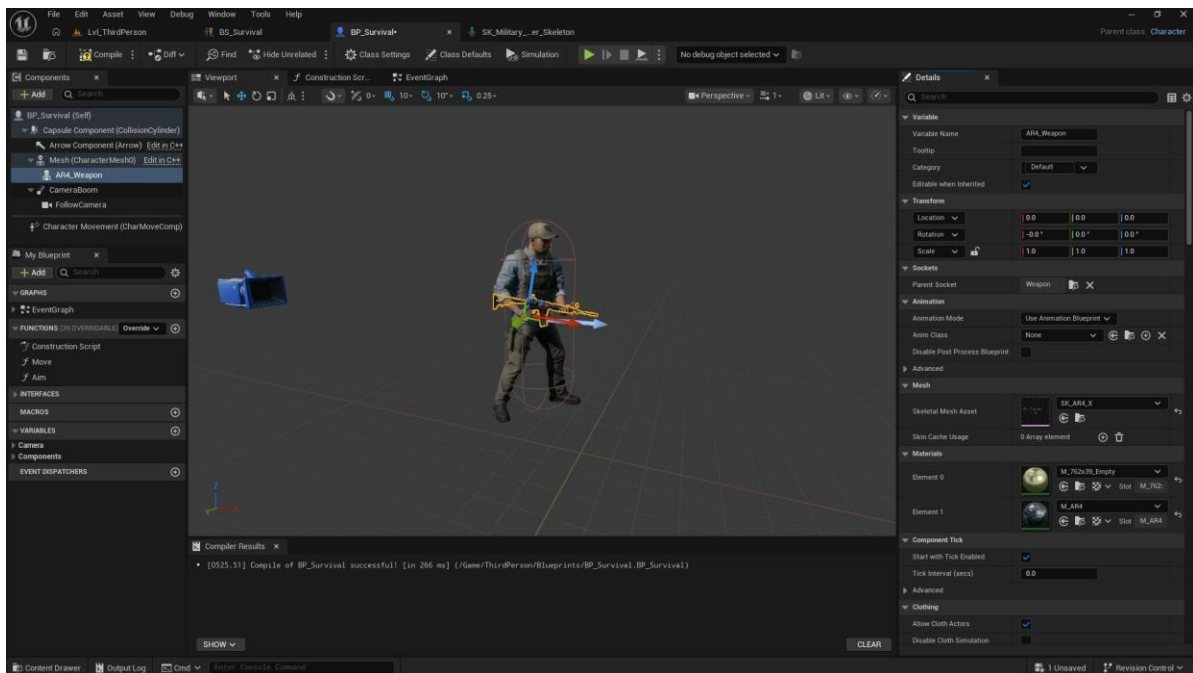


Figure 3.2: BP_Survival Blueprint Viewport showing the AR4 weapon mesh (SK_AR4_X) attached to the Weapon socket on the character's hand. The Details panel confirms the Parent Socket as Weapon and the Skeletal Mesh Asset as SK_AR4_X. Compiler result confirms successful compile in 266 ms.

Chapter 4

Animation System

4.1 Mixamo Animation Sourcing

Character animations were sourced from **Mixamo**, Adobe's online character animation library. Animations were selected and downloaded for the military character, including rifle walking, crouching, aiming, and running variants.

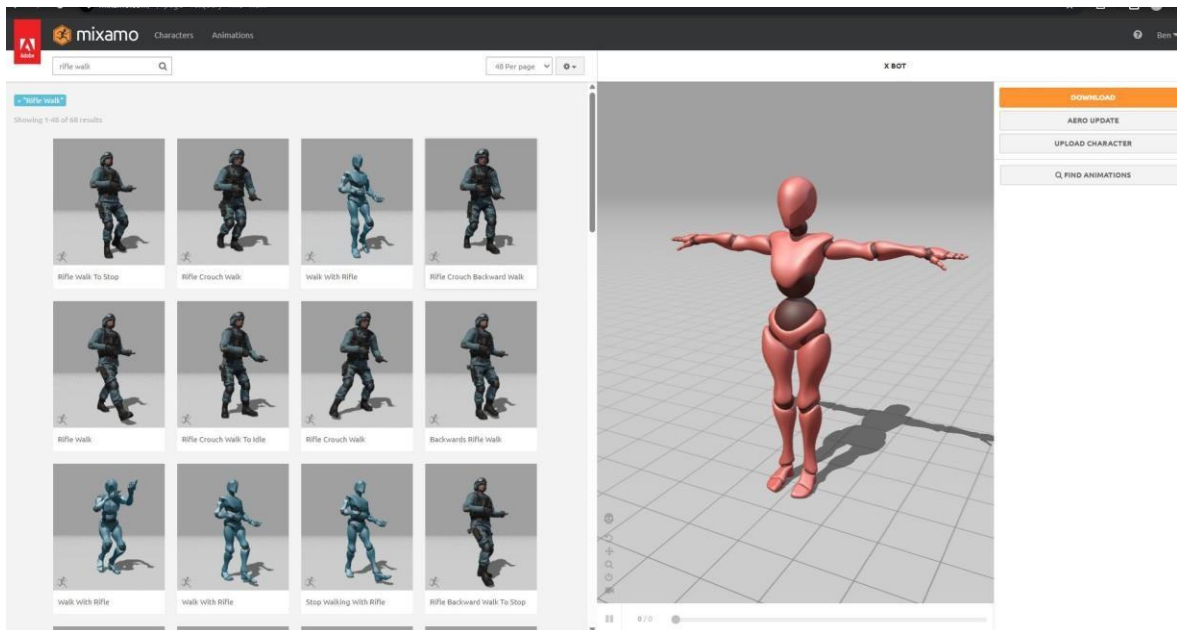


Figure 4.1: Mixamo web platform showing the *Rifle Walk* animation search results. Animations selected include Rifle Walk, Rifle Crouch Walk, Walk With Rifle, and Backwards Rifle Walk. The X BOT character is used as the base skeleton for download.

4.2 Animation Retargeting

Downloaded Mixamo animations were retargeted from the source SKM_QuantumCharacter skeleton to the project's military character skeleton (SK_Military_Character_Skeleton).

The retargeting workflow used Unreal Engine 5's IK Retargeter system.

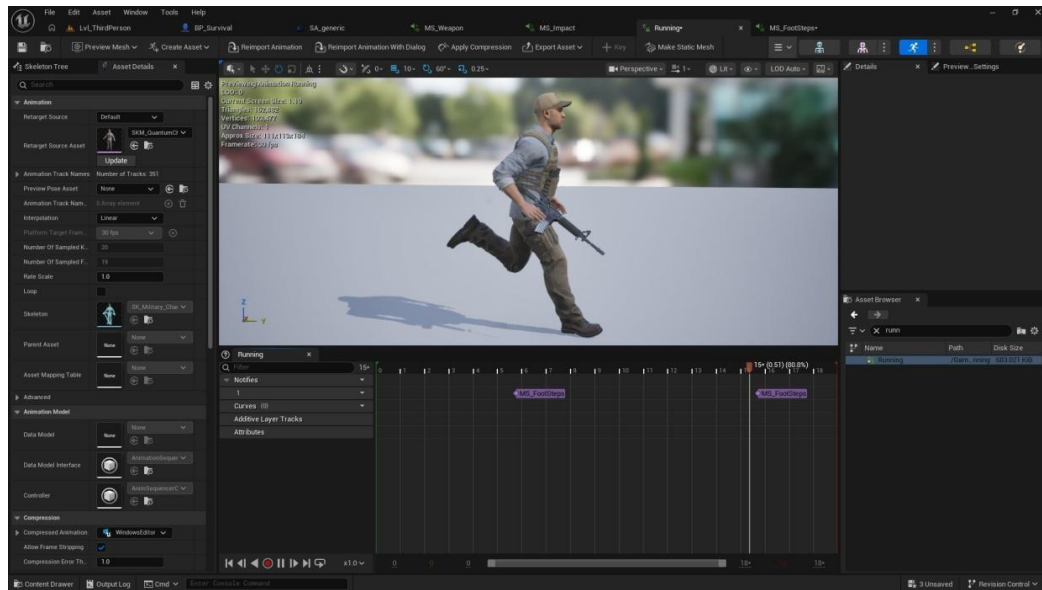


Figure 4.2: Running animation retarget in Unreal Engine 5. The Asset Details panel shows the skeleton set to SK_Military_Char, interpolation set to Linear, 20 sampled keys, and compression using WindowsEditor. The preview viewport shows the retargeted running pose at 30 FPS.

4.3 Animation Blueprint (ABP_Survival)

The ABP_Survival Animation Blueprint controls all player character animations. It uses an **Animation State Machine** with a Weapon State as the primary state, which transitions to an Aim state when the player aims.

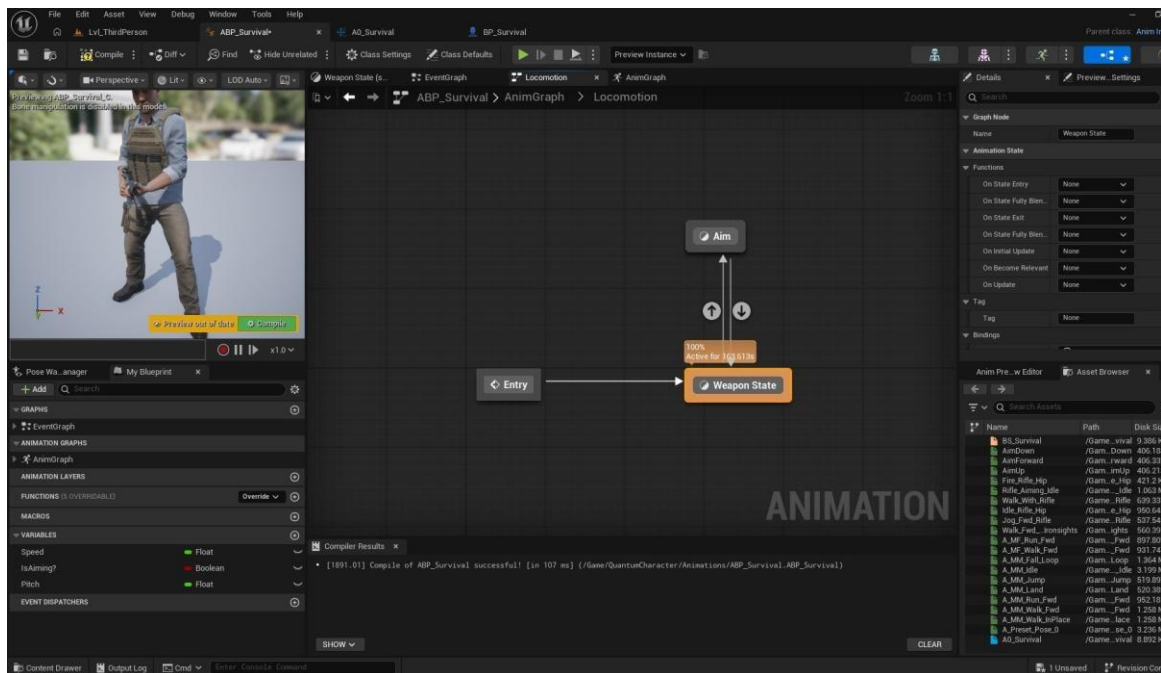


Figure 4.3: ABP_Survival Animation Blueprint. Left: preview shows the character in a walking pose with rifle. Centre: the Locomotion state machine graph showing Entry → Weapon State → Aim transitions. Right: Asset browser listing all animation sequences (BS_Survival, AimDown, AimForward, AimUp, Fire_Rifle_Hip, Walk_With_Rifle, Running, etc.).

4.3.1 Animation Variables

The Animation Blueprint exposes three variables used to drive state transitions and blend spaces:

Variable	Type	Purpose
Speed	Float	Controls walk/run blend
IsAiming?	Boolean	Triggers Aim state
Pitch	Float	Controls vertical aim offset

4.4 Aim Animation Sequence

The AimUp animation sequence was created as an *Additive Animation* (Rotation Offset Mesh Space) to layer aim direction on top of the base locomotion animation. It has 1 frame and 2 keys.

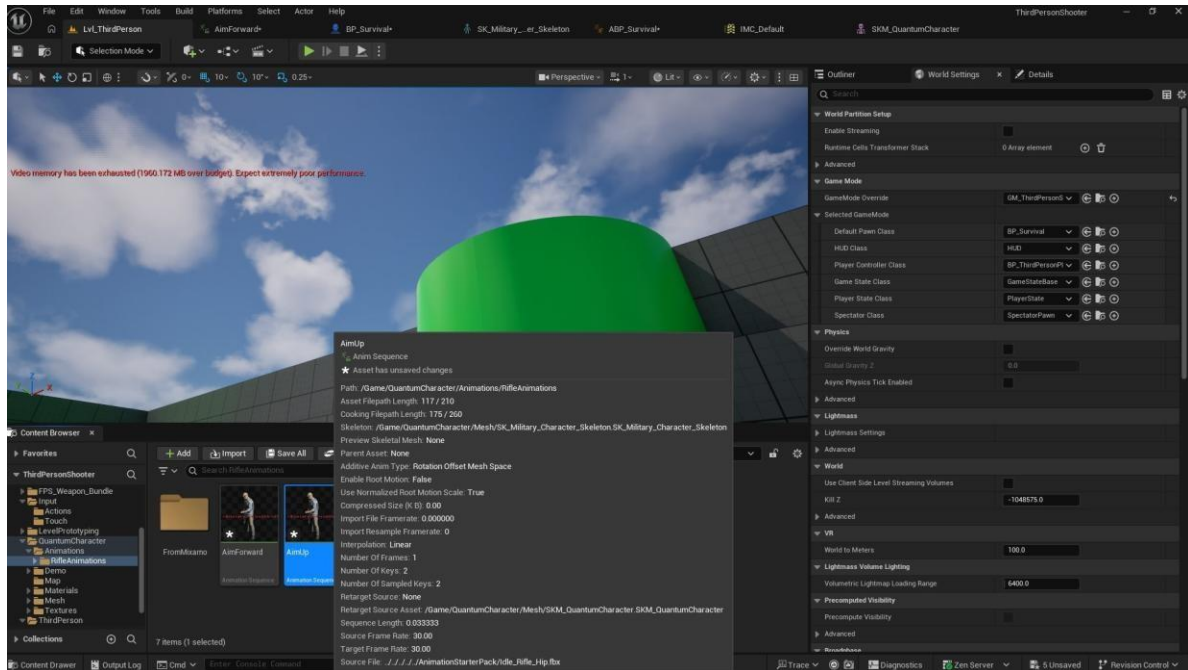


Figure 4.4: AimUp Animation Sequence details. Path: /Game/QuantumCharacter/Animations/RifleAnimations. Additive Anim Type: Rotation Offset Mesh Space. Retarget Source Asset: SKM_QuantumCharacter. Sequence Length: 0.033333s, Source Frame Rate: 30 FPS.

4.5 Running Animation with MetaSound Notifiers

The Running animation sequence includes **Animation Notifiers** (MS_FootSteps) at frames 7 and 15 to trigger the footstep MetaSound at the correct points in the running cycle.

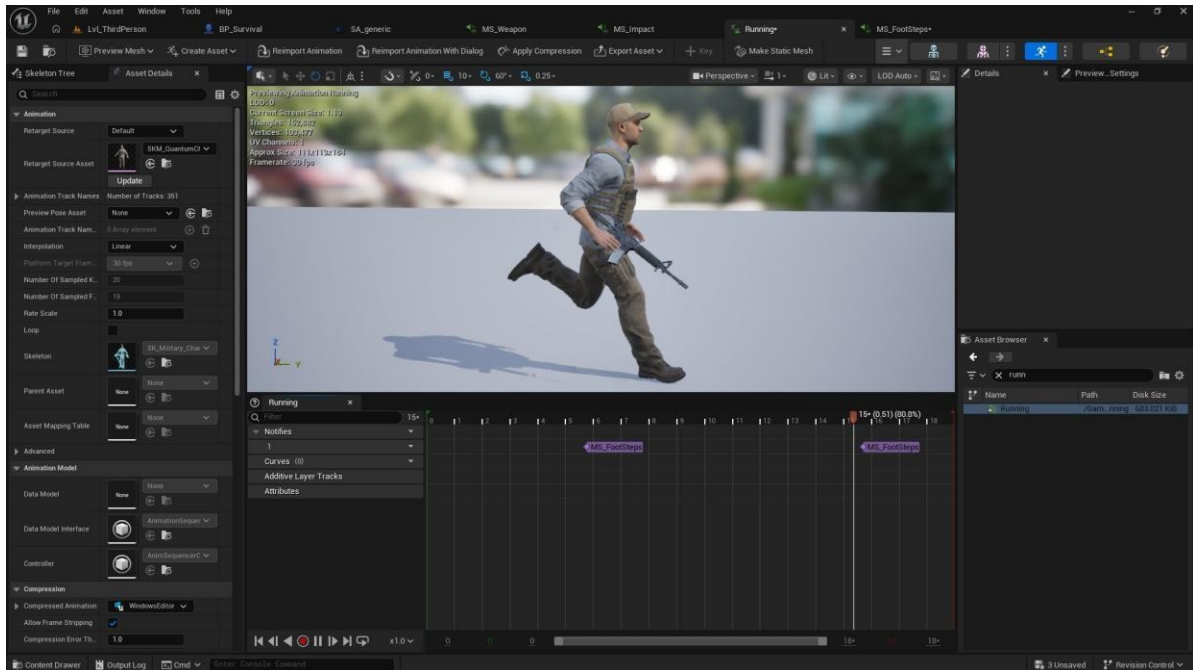


Figure 4.5: Running animation sequence editor. The timeline shows MS_FootSteps notify events placed at frames 7 and 15 to synchronise footstep audio with the running cycle. Preview shows the character mid-stride holding the rifle.

5.2 Crouch Mechanic

The crouch mechanic is implemented using the Enhanced Input Action `IA_Crouch`. On the *Started* event, the `IsCrouching?` Boolean variable is set to true, allowing the Animation Blueprint to blend into the crouch animation state.

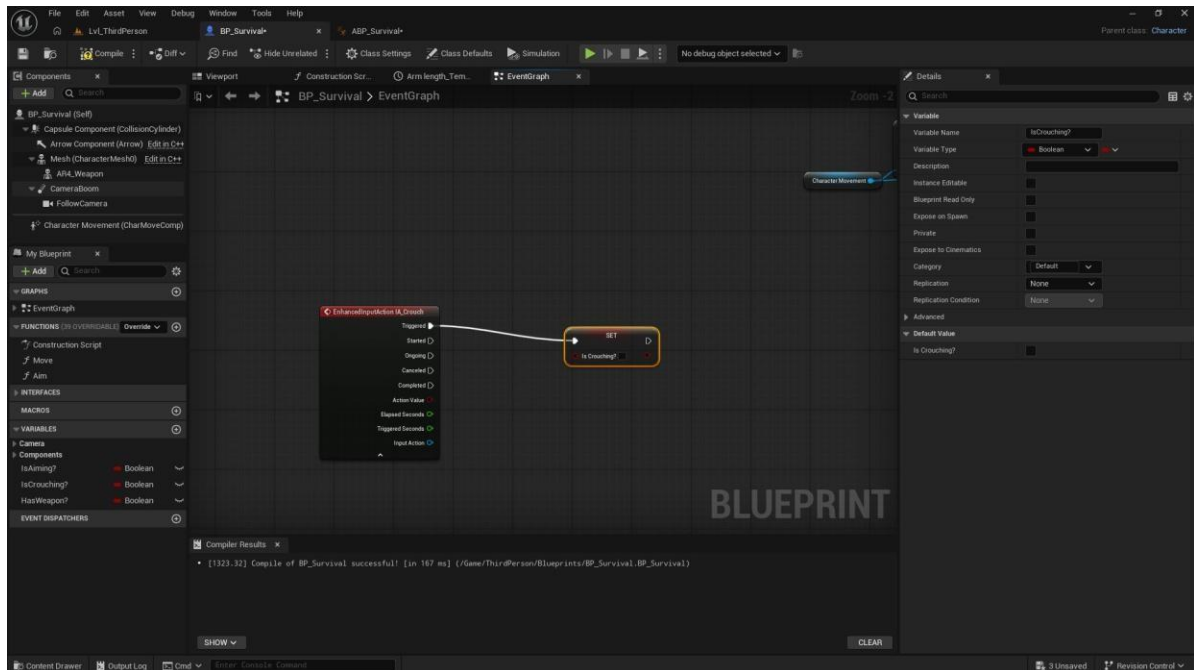


Figure 5.2: BP_Survival EventGraph – Crouch Blueprint. The EnhancedInputAction `IA_Crouch` node's *Started* pin connects to a SET node that updates the `IsCrouching?` Boolean variable. The Details panel on the right shows the variable's properties: Boolean type, Instance Editable disabled, Replication None.

5.3 Enemy AI Blueprint

The enemy AI is implemented in `BP_Survival_Enemy` with complex Blueprint logic for shooting detection, line traces, and applying damage. Two line trace channels are used to detect the player and trigger combat behaviour.

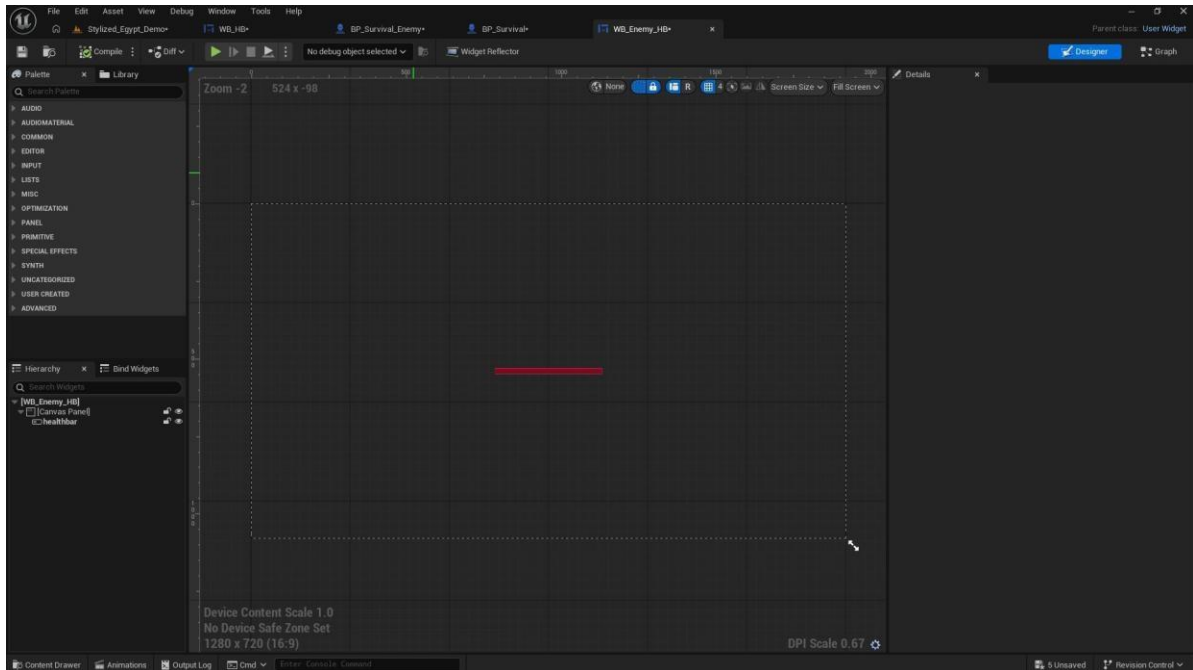


Figure 5.3: BP_Survival_Enemy EventGraph showing the full AI combat Blueprint. Key nodes include: Spawn Emitter Attached (muzzle flash), Play Sound at Location, two Line Trace By Channel nodes for hit detection, Break Hit Result, Apply Damage, and the HealthBar widget component. The compiler confirms a successful compile in 211 ms.

Chapter 6

Colour Models, Rendering and Visual Effects

6.1 Particle System – Muzzle Flash

The muzzle flash visual effect is implemented using Unreal Engine's **Cascade Particle System** (P_BackPack_MuzzleFlash). The system uses multiple emitters for a realistic gunshot effect.

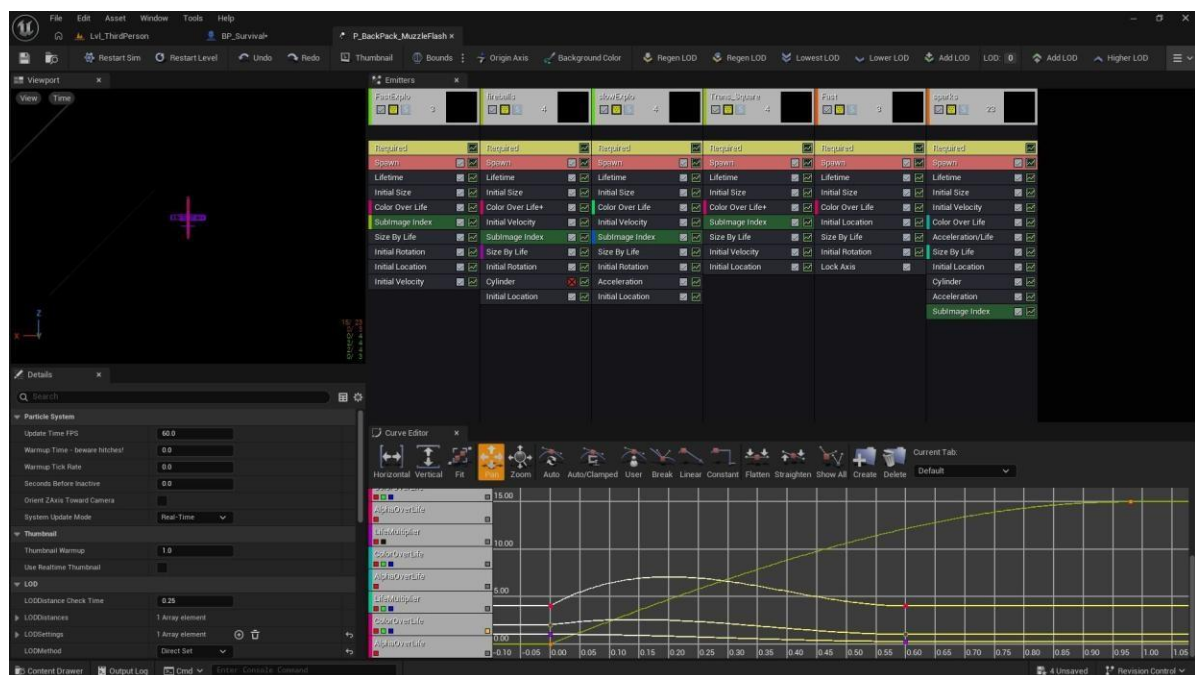


Figure 6.1: Cascade Particle System editor for P_BackPack_MuzzleFlash. Six emitters are visible: FastExplo (3 particles), Fireballs (4), SlowExplo (4), Trans_Square (4), Fast (3), and Sparks (23). The Curve Editor below shows alpha and colour curves over time. System Update Mode is Real-Time at 60 FPS.

6.1.1 Emitter Configuration

Each emitter uses a combination of modules:

- **Spawn:** Controls particle count and spawn rate
- **Lifetime:** Defines how long each particle lives
- **Initial Size / Size By Life:** Controls particle scale over its lifetime
- **Color Over Life:** Applies RGB colour transitions (orange to transparent)
- **Initial Velocity / Acceleration:** Drives particle motion
- **SubImage Index:** Enables sprite sheet animation for fire frames

6.2 Level Environment and Lighting

The game uses the **Stylized Egypt** asset pack for the primary level environment. The level features warm directional lighting to simulate an outdoor Middle Eastern setting.

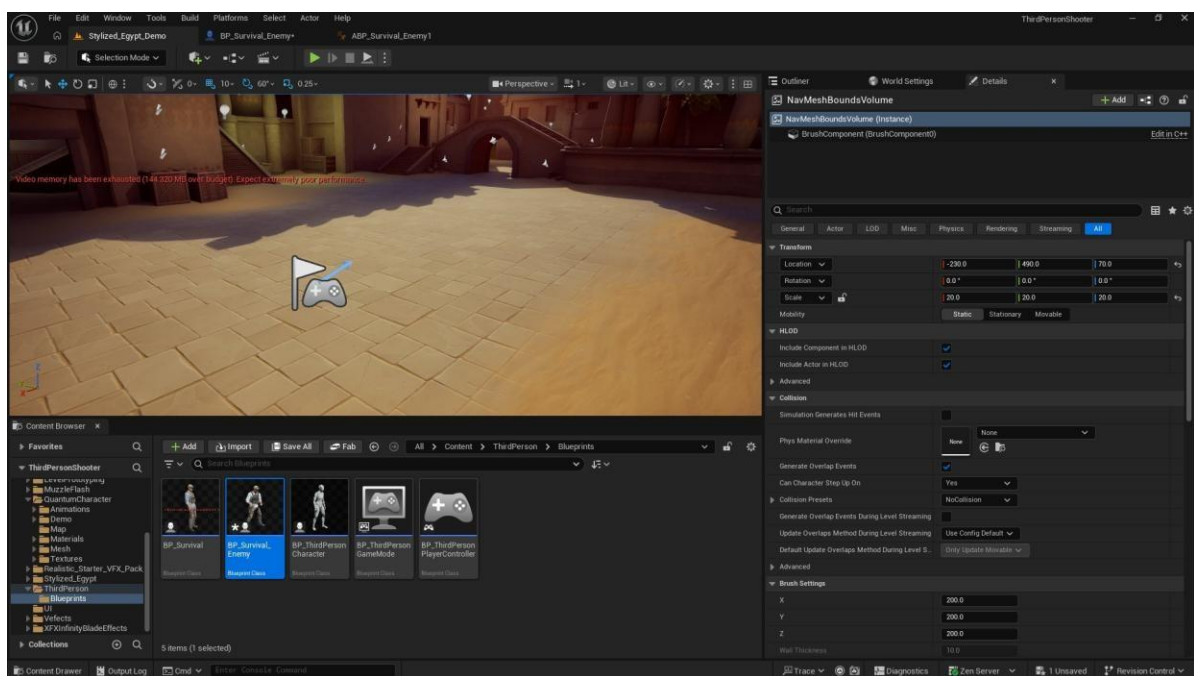


Figure 6.2: Stylized Egypt level in the Unreal Engine 5 editor. The viewport shows the sandy courtyard environment with archways, decorative pillars, and warm ambient lighting. The Outliner shows a NavMeshBoundsVolume placed for AI navigation. Blueprint assets visible in the Content Browser include BP_Survival, BP_Survival_Enemy, and associated controllers.

Chapter 7

User Interface Design

7.1 Player Health Bar (WB_HB)

The player's health bar is a UMG (Unreal Motion Graphics) widget containing a ProgressBar element (ProgressBar_1) styled in red. It is displayed in the player's HUD.

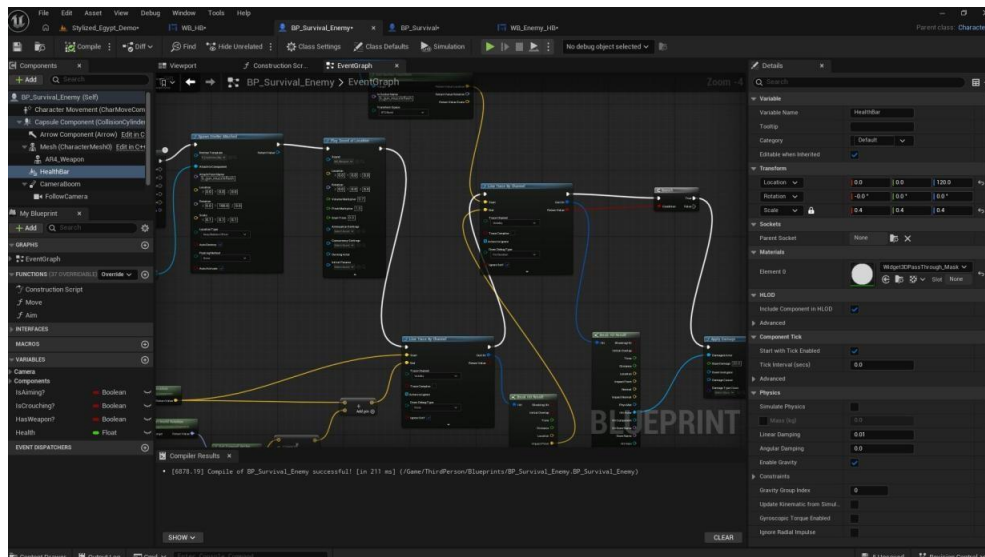


Figure 7.1: WB_HB widget designer. The hierarchy shows a Canvas Panel containing ProgressBar_1. The red progress bar is positioned at the bottom-right of the 1280×720 screen, representing the player's current health. DPI Scale: 0.67.

7.2 Enemy Health Bar (WB_Enemy_HB)

Each enemy character has a world-space health bar widget (WB_Enemy_HB) that floats above the enemy and diminishes as the enemy takes damage.

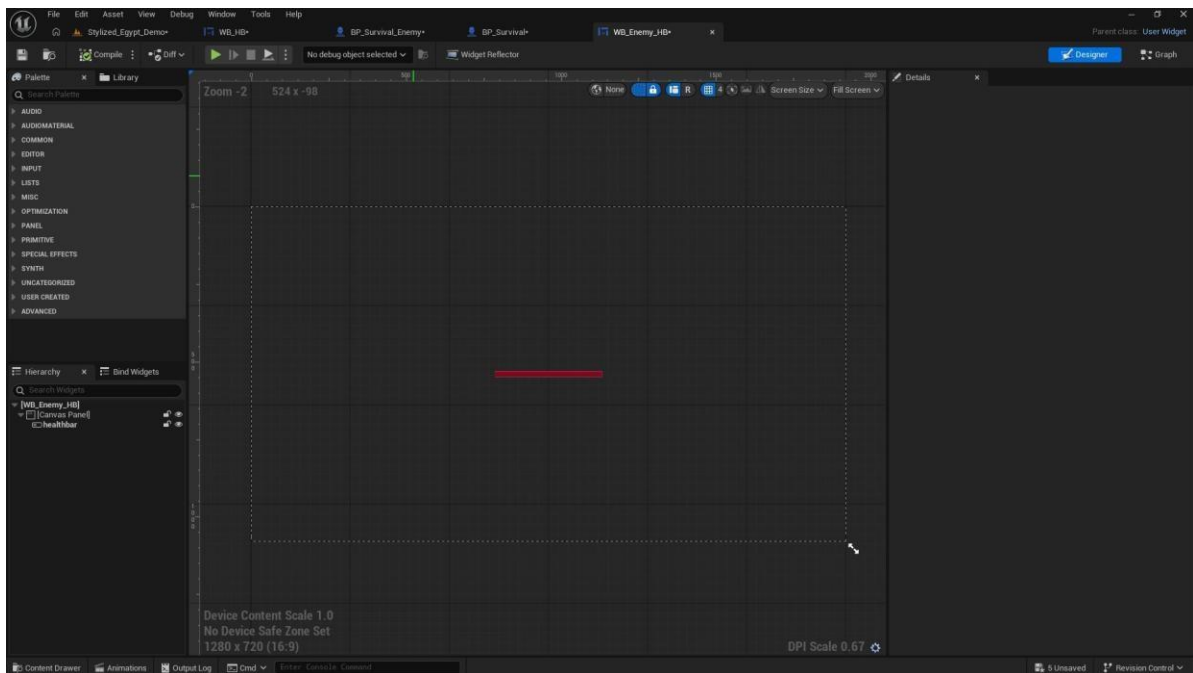


Figure 7.2: WB_Energy_HB widget designer. A Canvas Panel containing a healthbar progress bar element (red) is positioned centrally in the 1280×720 screen space, representing the enemy’s health. The widget is attached to the enemy character’s HealthBar component.

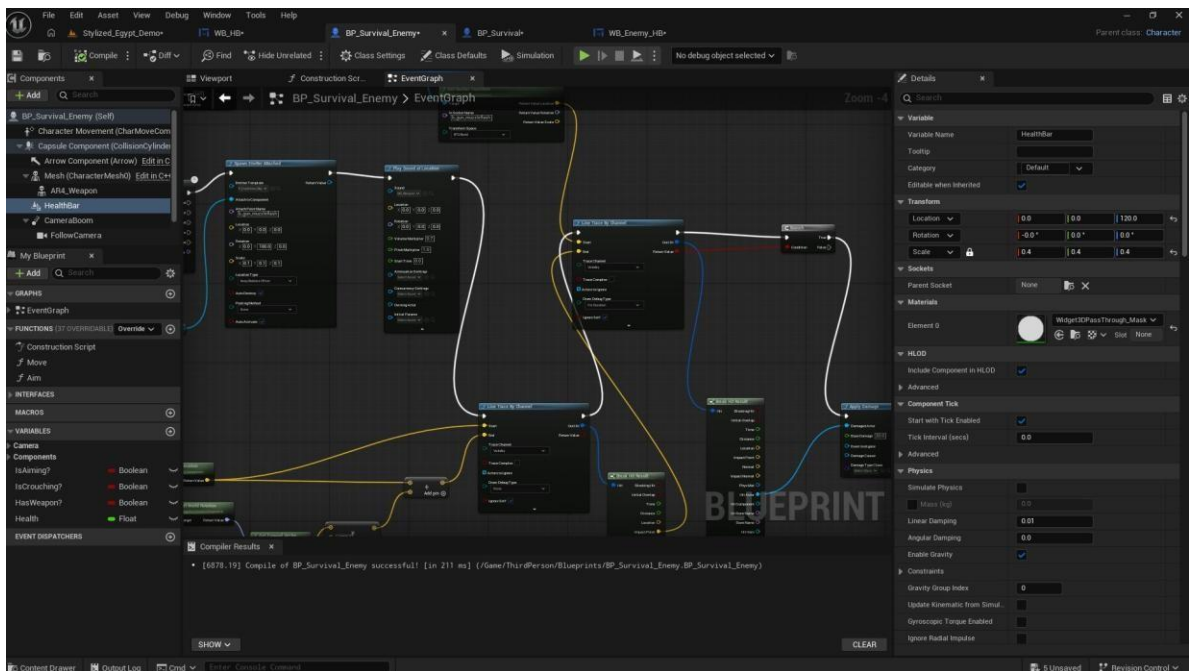


Figure 7.3: Enemy health bar widget in the UMG Designer. The healthbar element within the Canvas Panel is shown with the red colour indicating current health. Device Content Scale 1.0, resolution 1280×720 (16:9).

7.3 Crosshair Widget (WB_Crosshair)

The crosshair is a custom UMG widget (WB_Crosshair) rendered as a simple cross (+) shape using a Crosshair image element on a Canvas Panel. It is always centred on screen.

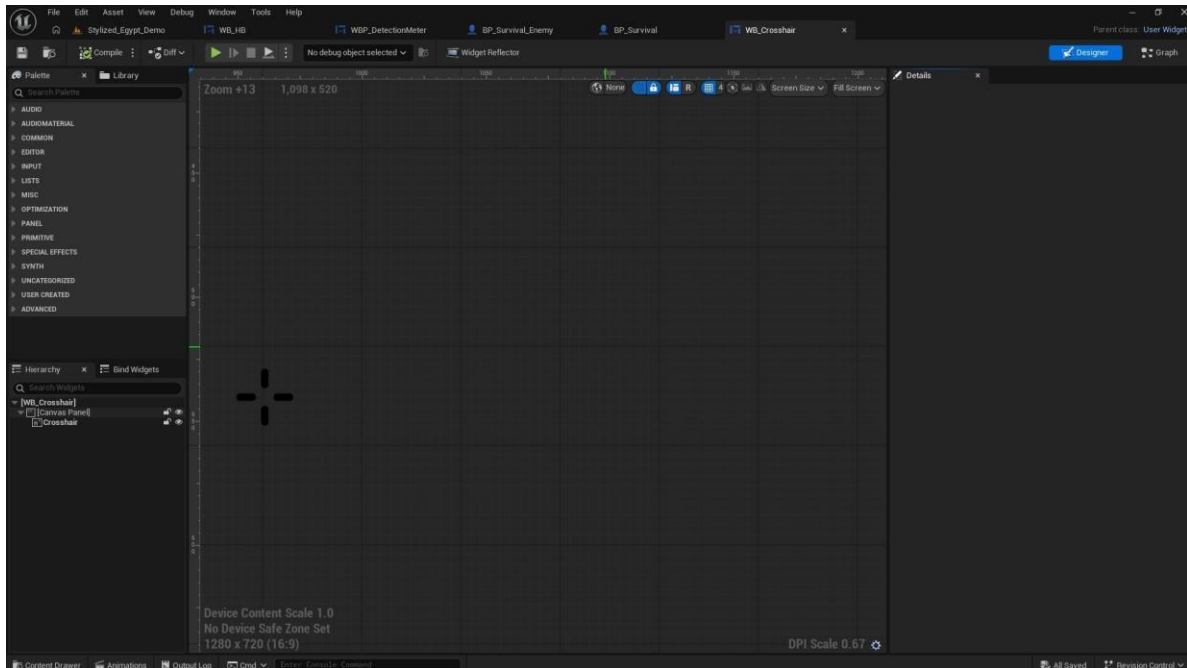


Figure 7.4: WB_Crosshair widget in the UMG Designer. A black crosshair image (Crosshair) is placed on a Canvas Panel and centred on the 1280×720 screen. The widget is rendered on top of the 3D scene during gameplay.

Chapter 8

Multimedia Integration

8.1 Audio System – MetaSound

The project uses Unreal Engine 5's **MetaSound** system for audio, which provides a node-graph-based audio processing pipeline. The footstep sound (MS_FootSteps) uses a **Random Float** node to randomise the pitch of each footstep between -4.0 and 4.0 , giving natural variation.

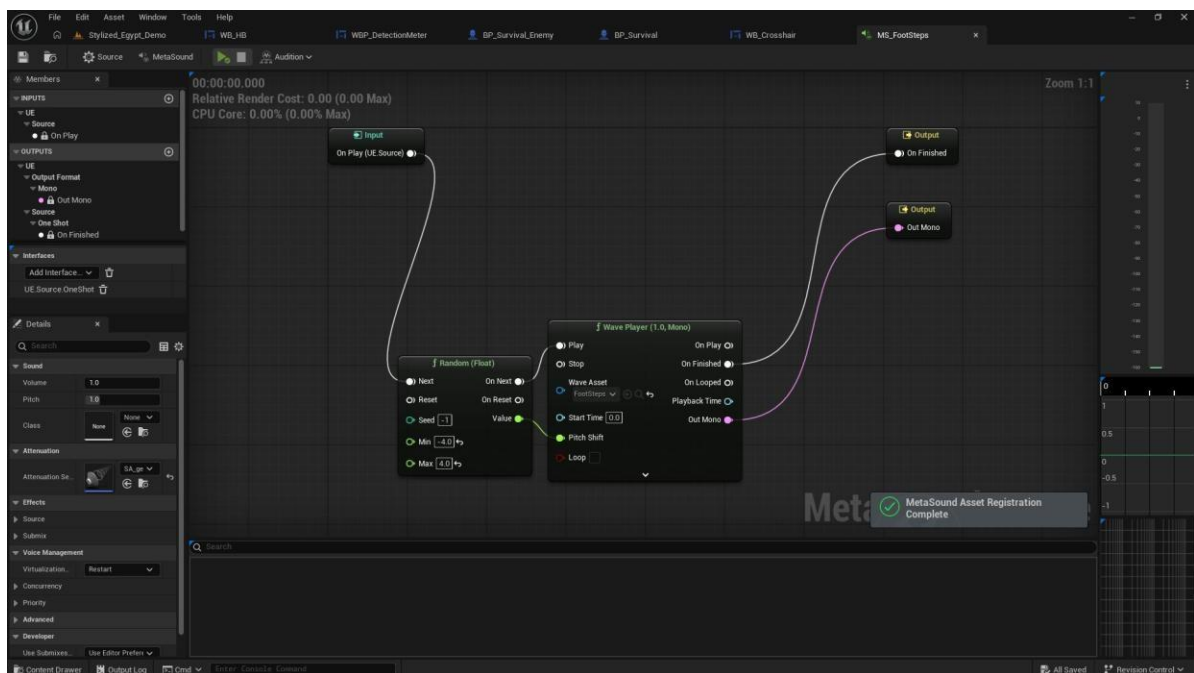


Figure 8.1: MetaSound graph for MS_FootSteps. The audio graph flows from *Input (On Play)* → *Random (Float)* (Seed: -1, Min: -4.0, Max: 4.0) → *Wave Player* (FootSteps wave asset, Pitch Shift driven by Random Float) → *Output (Out Mono)*. The MetaSound Asset Registration is confirmed as Complete.

8.1.1 Audio Architecture

Sound Asset	Type	Trigger
MS_FootSteps	MetaSound (One Shot)	Animation Notify on running/walking
MS_Weapon	MetaSound	Fire input action
MS_Impact	MetaSound	Line trace hit result
SA_generic	Sound Attenuation	Applied to all 3D sounds

8.1.2 Compression and Format Justification

Audio assets are stored as .wav source files and compressed at runtime using Unreal Engine's built-in audio compression pipeline (target platform: WindowsEditor). Mono output is used for footsteps to reduce memory usage. The MetaSound system processes audio at the CPU level, enabling real-time pitch randomisation without pre-baking multiple audio files.

8.2 Video and Animation Integration

Animation sequences imported from Mixamo are stored as .fbx files and imported into Unreal Engine as UAnimSequence assets. These are compressed using the **WindowsEditor** animation compression codec with *Allow Frame Stripping* enabled, reducing storage while maintaining playback quality at 30 FPS.

Chapter 9

Viewing, Clipping and Camera Control

9.1 Third-Person Camera Setup

The player camera is implemented using a **Spring Arm Component** (CameraBoom) attached to the character's capsule. The FollowCamera is a child of the spring arm.

- **Default Arm Length:** 400 units (third-person perspective)
- **ADS Arm Length:** 160 units (zoomed in for aiming)
- **Projection:** Perspective projection (built into Unreal's camera component)
- **Lag:** Camera lag is disabled for responsive player control

9.2 Window-to-Viewport Mapping

Unreal Engine 5 performs window-to-viewport mapping automatically via its rendering pipeline. The game runs at a target resolution of 1280×720 (16:9) as defined by the UMG widget canvas. The DPI Scale of 0.67 is applied to UI elements to ensure correct scaling at this resolution.

9.3 Scene Clipping

Unreal Engine's renderer applies a near clip plane (default 10 units) and a far clip plane defined by the camera's view distance settings to cull geometry outside the view frustum. The Cascade Particle System's LOD distance check time of 0.25s manages particle system level of detail.

Chapter 10

2D and 3D Transformations

10.1 Skeletal Mesh Transformations

All 3D objects in the scene are positioned using Unreal Engine's transformation system, which applies Translation, Rotation, and Scale via 4×4 homogeneous transformation matrices.

For example, the NavMeshBoundsVolume (see Figure 6.2) is transformed as:

- **Location:** $(-230, 490, 70)$
- **Rotation:** $(0^\circ, 0^\circ, 0^\circ)$
- **Scale:** $(20, 20, 20)$

The weapon socket on hand_r applies a local-space transformation:

- **Relative Location:** $(-5.66, 2.99, -0.97)$
- **Relative Rotation:** $(8.99^\circ, 0.0^\circ, 174.27^\circ)$
- **Relative Scale:** $(1.0, 1.0, 1.0)$

10.2 Camera Arm Transformation

The aim mechanic applies a real-time transformation to the CameraBoom component's Target Arm Length property. This is a 1D scaling transformation along the local Z-axis of the spring arm, transitioning from 400 to 160 units when aiming (see Figure 5.1).

10.3 Projection

The game uses **perspective projection** (the default for Unreal's UCameraComponent), where objects appear smaller as they recede from the camera. This is implemented via a perspective projection matrix internally applied by the engine's rendering pipeline.

Chapter 11

Graphics Algorithms

11.1 Line Drawing – Line Trace Algorithm

The project implements a software **line drawing algorithm** through Unreal Engine's **Line Trace By Channel** Blueprint node, used in the enemy AI shooting logic (see Figure 5.3). This performs a ray-cast from the enemy's weapon muzzle in the forward direction, detecting the first object along the ray within the trace channel (Visibility or ECC).

The underlying algorithm is equivalent to a 3D parametric line:

$$P(t) = P_0 + t \cdot \vec{d}, \quad t \in [0, T_{max}]$$

where P_0 is the start point (muzzle location), $\vec{d}$ is the normalised direction vector, and T_{max} is the maximum trace distance.

11.2 Circle Drawing – Particle Emitters

The Cascade Particle System uses a **Cylinder** emitter module (visible in the P_BackPack_MuzzleFlash system, Figure 6.1) to emit particles in a circular/cylindrical pattern. Internally, this uses a polar coordinate-based circle equation:

$$x = r \cos \theta, \quad y = r \sin \theta$$

to distribute initial particle positions around the muzzle origin.

11.3 Bézier Curves – Animation Curves

The Cascade Particle System's Curve Editor (see Figure 6.1) uses **cubic Bézier curves** to define how particle properties (Alpha, LifeMultiplier, ColorOverLife) change over the particle's lifetime. The curves are defined by four control points P_0, P_1, P_2, P_3 :

$$B(t) = (1 - t)^3P_0 + 3(1 - t)^2tP_1 + 3(1 - t)t^2P_2 + t^3P_3$$

Chapter 12

Team Collaboration

12.1 Task Distribution

Team Member	Responsibilities
Benjamin Acheampong	Project Lead, Game Mode Setup, Level Design
Yoofi Egyin Brew-Hammond	Character Blueprint, Input System
Jeremy Bradley Aryee Arthur	Animation Blueprint, Retargeting
Amazing Naa Dedei Quaye	UI/UMG Widgets (Health Bar, Crosshair)
Edinam Kumordjie	Enemy AI Blueprint, Line Trace Logic
Albert Afful	Particle Systems, Visual Effects
Stanley Agambire	MetaSound Audio Integration
Bayaka Melvin Soale Braimah	Weapon Attachment, Socket Setup
Samuel Nii Ayitey Aryee	Camera System, ADS Mechanic
Erica Dzifa Agbana	Technical Report, Documentation

12.2 Academic Integrity

All work presented in this report was produced originally by the team. External assets used (Mixamo animations, Stylized Egypt asset pack) are used under their respective licences. All Blueprint logic was written by team members. This report has been prepared in compliance with Academic City University's academic integrity policy.

Chapter 13

Conclusion

This project successfully demonstrates the integration of computer graphics and multimedia concepts within a real-time 3D game environment built in Unreal Engine 5. The team implemented and documented the following key course requirements:

- **Graphics Algorithms:** Line trace (ray casting), circular particle emission, and Bézier curve-driven animation curves
- **2D/3D Transformations:** Skeletal socket transforms, camera arm scaling, and world-space object placement
- **Viewing and Clipping:** Perspective projection, frustum culling, and viewport mapping
- **Colour and Rendering:** RGB-based particle colour curves, texture-mapped character and weapon materials, and real-time lighting
- **Animation:** Keyframe and additive animation sequences, blend spaces, state machines, and notifiers
- **Multimedia Integration:** MetaSound audio graphs with real-time pitch randomisation, animation-notifier-triggered audio, and FBX-based video/animation import
- **User Interface:** UMG-based HUD with health bars and crosshair widget

The final product is a functional third-person survival shooter demo environment that serves as a solid foundation for further gameplay development.

Appendix A

Screenshot Index

Figure	Screenshot File	Content
2.1	Screenshot_2026-04-03_160226	Game Mode Blueprint
2.2	Screenshot_2026-04-03_161911	Level & Animation BP
3.1	Screenshot_2026-04-03_170049	Weapon Socket
3.2	Screenshot_2026-04-03_170320	AR4 Weapon Component
4.1	Screenshot_2026-04-03_171055	Mixamo Animations
5.1	Screenshot_2026-04-03_183238	Aim Blueprint
4.4	Screenshot_2026-04-03_203106	AimUp Sequence
4.3	Screenshot_2026-04-04_000140	ABP_Survival
4.2	Screenshot_2026-04-04_140215	Running Retarget
6.1	Screenshot_2026-04-04_120752	Muzzle Flash Particles
6.2	Screenshot_2026-04-05_004808	Egypt Level
5.3	Screenshot_2026-04-05_031409	Enemy AI Blueprint
7.1	Screenshot_2026-04-05_032038	Player Health Bar
7.4	Screenshot_2026-04-12_174018	Crosshair Widget
8.1	Screenshot_2026-04-12_174044	MetaSound Graph